Assignment 4. Patient Zero

An outbreak, a genome and a list of people to test: dynamic programming and string algorithms in one investigation.

Course	Design and Analysis of Algorithms
Topics	Dynamic programming (edit distance, longest common subsequence, DP on a tree), string matching (prefix function, KMP, Rabin-Karp rolling hash)
Weight	100 points: assignment 50, defense (a quiz in Moodle) 50
Plan	About 4 to 5 hours of work in total. Missions 1 and 2 are the shortest, Mission 3 the longest
Language	Java 17 or newer. No other language and no string-search or hashing libraries for the missions.
Submission	Link to your GitHub repository, uploaded in Moodle before the deadline.

1. The task

A new virus appeared. You are the analyst of the lab. You have two strains of the virus: the old one (A) and the new one (B). You have a reference genome, a gene that you must find, and the tree of infections: who infected whom. The questions are the ones a real lab asks, and every one of them is an algorithm of the course.

- How much did the virus change? (edit distance, longest common subsequence)
- Where is the dangerous gene in the genome? (KMP) Which parts of the new strain were copied from the old one? (Rabin-Karp)
- Whom do we test, so that every infection link is covered and the test costs are the smallest? (dynamic programming on a tree)

Every student has personal files made from the student ID, so every number in your output is your own. The starter project has a file START_HERE.md: read it first. Your program also writes a page output/outbreak.html, the lab console: open it in a browser. It recomputes every mission on your data and shows with animations how the algorithms work. It never shows the correct answer, only a tick or a cross for each of your 9 lines, so a cross means: watch the animation and find where your code behaves differently.

2. Your input files

The starter project contains tools/MakeVariant.java. Run it with your student ID:

```
java tools/MakeVariant.java YOUR_ID data
```

It creates four files in the folder data. Type the ID exactly as in Moodle (012345 and 12345 give different files). The letters in all sequences are A, C, G, T. Positions are counted from 0.

File	Content
strains.txt	Two lines: "A SEQUENCE" and "B SEQUENCE". Strain A is the old virus, B is the new virus. About 900 to 1300 letters each.
genome.txt	One line: the reference genome. About 24 000 to 32 000 letters.
markers.txt	One line "marker WORD": the gene you look for.
tree.txt	First line n: the number of people, numbered 0 to n-1. Then n-1 lines "x y": one of the two infected the other (the direction is not given, root the tree in person 0 yourself). The last line has n numbers: the cost of testing person 0, 1, ..., n-1.

A tiny example (examples/tiny in the starter project). You can solve it on paper:

```
strains.txt                genome.txt          markers.txt         tree.txt
A ACGTACGTACGTAC           ACGACGACGT          marker ACGAC        6
B ACGTACGTACGTAG                                                   0 1
                                                                   0 2
                                                                   1 3
                                                                   1 4
                                                                   2 5
                                                                   3 2 4 1 5 2
```

3. Program and output

Every mission has its own file in src, Main.java is ready and prints the lines. Compile and run:

```
javac -encoding UTF-8 -d out src/*.java
java -cp out Main
```

Without arguments the program reads the files in data. The command java tools/Check.java compiles the program and checks it on three examples with the right output. The program prints the 9 lines below, each as "name: whole number", in this order. Other lines are allowed. The program must not wait for keyboard input and must finish on your data in under one minute.

Output line	Meaning	Mission
Mutations	The smallest number of single-letter changes (replace a letter, insert a letter, delete a letter) that turn strain A into strain B.	1
Shared core	The length of the longest common subsequence of A and B: letters that occur in both strains in the same order, not necessarily next to each other.	2
Marker hits	The number of places in the genome where the marker occurs. Occurrences may overlap and all of them count.	3
First hit	The position of the first occurrence of the marker (from 0), or -1 if there is none.	3
Border length	The length of the longest proper prefix of the marker that is also a suffix of the marker (the last value of the prefix function).	3
Shared words	The number of positions i in A such that the 12 letters A[i..i+11] also occur somewhere in B. Count positions, not different words.	4
Tests needed	Choose people to test so that in every infection link at least one of the two people is tested. The smallest number of people for which this is possible.	5
Cheapest testing	The same, but every person has a cost of testing: the smallest total cost.	5
Longest chain	The number of links on the longest chain of infections, that is the longest path in the tree.	5

Output for the tiny files above, to check your reading of the rules:

```
Mutations: 1        Shared core: 13       Marker hits: 2
First hit: 0        Border length: 2      Shared words: 2
Tests needed: 2     Cheapest testing: 6   Longest chain: 4
```

4. Missions (50 points)

Mission	What to do	Points
1. Mutations	Edit distance with a table d[i][j]. README: what d[i][j] means and how it comes from smaller cells.	8
2. Shared core	Longest common subsequence with the same kind of table. README: how it differs from the longest common substring.	6
3. The marker	Prefix function (the LPS array of the lecture) and KMP: the pointer in the text never goes back. Do not use indexOf, contains, regular expressions or split. README: what the prefix function tells you, and what Border length means for your marker.	10
4. Copied words	Rabin-Karp with a rolling hash: the hash of the next window is computed from the previous one in O(1). When the hashes are equal, compare the letters. README: why you must compare, and what happens to the time when many hashes are equal.	9
5. Whom to test	DP on the tree rooted in person 0: for every person the cheapest cost when the person is tested and when the person is not tested. Rebuild who is tested (the picture needs it). Longest chain by two longest ways down. README: the two states, and why the greedy choice does not work.	10
6. Git and README	Nothing to code. A commit after every mission with a message that says what was done (2 points). The questions in README answered in your own words and the section AI use filled in (2 points).	4
7. Picture, timing, complexity	Nothing to code. The screenshot of output/outbreak.html is in the repository (1 point). Run the ready class Bench and put the times into README with two or three sentences on how the time grows (1 point). Fill in the table of time and space complexity of the algorithms you wrote (1 point).	3

Do the missions in order. The deadline is set in Moodle: upload the link to your repository there before it.

How long it takes: about 4 to 5 hours in total. Mission 1 about 40 minutes, 2 about 30 minutes, 3 about 1 hour, 4 about 45 minutes, 5 about 1 hour, 6 and 7 about 45 minutes. Start early and do not leave Mission 3 for the last evening. The file

docs/Hints.md in the starter project gives, for every mission, the idea, the usual trap and how to find the mistake with the lab console.

How the points are given. Every mission is checked on your data and on the data of other students, with the same code. A mission gets its points when the output is right, the idea of the lecture is used (for example KMP and not indexOf) and the README answer is yours. A mission with a right output but a hand-typed number or a forbidden method gets zero. A mission that is partly right gets part of its points: for example the prefix function right and the search wrong in Mission 3.

5. Submission

Submit the link to your GitHub repository in Moodle, for example https://github.com/username/patient-zero. The repository must be public.

- All source code is in the repository. Work on the main branch and commit after every mission with a message that says what was done (at least 5 commits). The main branch compiles.
- The report is the README.md of the repository: your ID, the output of your program, the answers, the AI use note, the timing table and the complexity table. The starter project already has the template.
- The four files in data and the screenshot output/outbreak.png are in the repository.

A submission without a working link or without the README report is not accepted.

6. Defense (50 points)

The defense is a quiz in Moodle: 20 questions, 30 minutes, 2.5 points each, 50 points in total. It is held in class on a date announced separately. Every student gets different questions from a bigger bank. You may use paper and a pen; phones, messages and AI tools are not allowed.

What the questions are about. Three questions are very short warm-up questions that check that you have read the assignment. Three are small calculations by hand: the edit distance or the LCS of two short words, the prefix function of a short marker or the number of its occurrences, and the cheapest testing on a tree of five or six people. Each calculation question explains the task in its text. The other fourteen questions are theory about the algorithms of the assignment: dynamic programming, edit distance and LCS, the prefix function and KMP, Rabin-Karp, and trees and the DP on a tree. They ask what a table or a function means, why a method is correct, what it costs in time and memory, and what goes wrong if a step is left out. The answers are given in several ways: choose one answer of four, select all the right answers, match items, true or false, or type a whole number.

How to prepare: read the file docs/Assignment4_defense_prep.pdf. It lists the types of questions with a worked example of each calculation, the theory points to know, a short glossary and a few practice questions with answers. Then repeat each algorithm by hand on the tiny example from examples/tiny and watch its animation in the lab console. If you wrote and understood the code, the quiz is easy.

7. Rules

- Your own work only. Copied code, a shared solution or unauthorized collaboration gives zero for the assignment. Code or text taken from a book or a website must be cited in the README with the source.
- AI tools (ChatGPT, Copilot and similar) may be used as a helper: to explain an idea, to understand an error, to review your code. The syllabus requires you to say so in writing: the README has a section "AI use" where you name the tool and what you used it for (write "none" if you did not use one). AI-generated work submitted without changes or without this note is academic dishonesty. You must understand every line you submit; the quiz will show it.
- Deadlines. Submit the link in Moodle before the deadline: email and messengers are not accepted. A late submission or a late defense gets zero. An extension is possible only if you ask before the deadline and have a documented reason.
- Your code must compute the answers. A number typed into the code, a result read from the lab console or a condition that checks the ID gives zero for the assignment. Your program is also run on the data of other IDs.
- Do not use ready-made string-search or hashing libraries (indexOf, contains, regular expressions, split for searching). The Java standard library (collections, arrays, HashMap, ArrayDeque) is allowed.